

ROCKET CREDIT PROJECT – CHAT TRANSCRIPT

=====

Prepared: 11 September 2026

Scope: User and assistant messages about the Rocket Credit project.
Internal system messages and tool output are excluded.

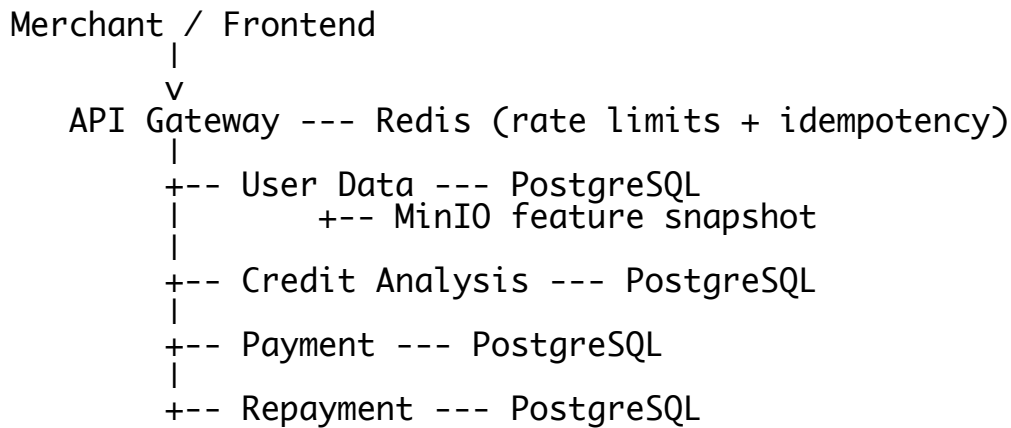
USER

explain me whole project

ASSISTANT

PROJECT OVERVIEW

Rocket Credit is a diploma/prototype BNPL platform: a merchant submits a purchase, the system evaluates the buyer's creditworthiness, creates a merchant payment, and schedules repayments.



Repository structure

- * demo-repository: React/Vite marketing site and static customer dashboard.
- * rocket-credit-gateway: Public /checkout endpoint and workflow orchestration.
- * rocket-credit-user-data: Users, transaction history, payment tokens and derived credit features.
- * rocket-credit-analysis: Rule-based credit scoring and decision audit.
- * rocket-credit-payment: Merchant transfer records.
- * rocket-credit-repayment: Repayment plans, installments and scheduled charges.
- * rocket-credit-claimcheck: Shared Java library for storing large feature payloads in S3/MinIO.
- * rocket-credit-deployment: Docker Compose, OpenAPI contracts and smoke-test script.

These are effectively several sibling repositories rather than one Maven monorepo.

Complete checkout flow

The main endpoint is POST /checkout. A request contains partner and payment identifiers, amount, currency, installment duration, buyer information, transaction history and purchased items.

1. Idempotency check

Redis stores the result under the partner and payment ID. Concurrent duplicates receive HTTP 409; completed requests are replayed for 24 hours.

2. Resolve the user
User Data searches by partner user ID, then email. It creates or updates the user and imports supplied transaction history.
3. Compute credit features
User Data derives values such as order count, average order value, income estimate, credit limit and Rocket repayment history.
4. Create a feature claim
The feature JSON is stored in MinIO. Only a bucket/key reference is passed through the Gateway. This is the claim-check pattern.
5. Make the credit decision
Credit Analysis downloads the feature snapshot, calculates a score and stores an explainable audit.
6. Stop if denied
The Gateway returns approved: false and hides internal scoring details behind CREDIT_DENIED.
7. Create payment
Payment writes a transfer record and immediately marks it TRANSFERRED.
8. Create repayment plan
Repayment divides the total into equal monthly installments. Rounding leftovers are distributed one cent at a time.
9. Validate and return schedule
The Gateway verifies installment count, dates and total amount before returning the result.

Credit scoring

Final score =
 Rocket history x 50%
 + Partner history x 30%
 + Purchase amount x 20%

The factors are:

- * Rocket history: on-time ratio, late payments, active plans and customer tenure.
- * Partner history: order count and average order value.
- * Amount: how much of the calculated borrowing capacity the purchase uses.

A purchase is approved when KYC passes, the score is at least 0.60, and the requested amount is within capacity. Scores between 0.50 and 0.60 produce an internal review outcome. Policies and thresholds are versioned in PostgreSQL.

Every decision stores its score, reason codes, individual factor scores and applied weights.

The current input data is still synthetic:

- * Missing KYC effectively defaults to passed.
- * Rocket on-time ratio is hardcoded to 0.9.
- * Late payments and active plans default to zero.
- * Income and credit limit are estimated from average monthly transaction volume.

Therefore this is currently deterministic rule-based scoring, not a trained AI model.

Individual services

Gateway

Besides orchestration, the Gateway provides Redis-backed rate limiting, idempotent checkout processing, retries for User Data calls, circuit breakers, concurrency limits, downstream URL pinning and coarse public errors. HMAC and mutual-TLS filters exist but are commented out.

User Data

User Data owns customer identity and transaction data. It uses Spring Boot, JPA, PostgreSQL and Flyway. Feature overrides can be supplied through `/users/{id}/features` for development and scoring experiments.

Payment

Payment creates idempotent merchant-transfer records. There is no real payment provider yet: it uses provider NONE, creates a local reference and immediately reports success.

Repayment

Repayment creates monthly schedules. A scheduler runs daily at 03:00 UTC, charges due installments and retries failures up to three times. The current fake provider always succeeds.

Frontend

The frontend provides a landing page, How It Works page, business marketing page, About page, static account dashboard and interactive payment calculator. It currently has no fetch/API integration. The dashboard and company statistics are hardcoded. Its calculator also adds interest for longer plans, while the backend simply splits the original total.

Infrastructure

Docker Compose starts:

- * Gateway: 8080
- * User Data: 8081
- * Credit Analysis: 8082
- * Payment: 8083
- * Repayment: 8084
- * Redis: 6379
- * MinIO: 9000, console 9001
- * Separate PostgreSQL databases: host ports 5433-5436

Java services use Spring Boot; Credit Analysis uses FastAPI, SQLAlchemy and Alembic.

Current limitations

- * No trained AI, digital-footprint service or bank-data adapter.
- * Frontend is not connected to the backend.
- * Payment and card charging are simulated.
- * HMAC/mTLS authentication is disabled; User Data permits all routes.
- * No automated test files are present.
- * Several OpenAPI contracts have drifted from their implementations.
- * No compensation workflow exists if payment succeeds but repayment-plan creation fails.

- * Credit Analysis defines /_health, while Compose checks /actuator/health.
- * Notification, Kafka events, partner SDK, portal and webhooks are planned but inactive.

The future architecture in FULL_WEB_APPLICATION_PLAN.md should be read as a roadmap rather than the current implementation.

Running it

Backend:

```
cd rocket-credit-deployment
docker compose up --build
```

Frontend separately:

```
cd demo-repository
npm install
npm run dev
```

The frontend runs as a standalone demo and does not automatically communicate with the backend.

USER

what kind programming languages where used

ASSISTANT

The project uses:

- * Java – Gateway, User Data, Payment, Repayment, and Claim Check services.
- * Python – Credit Analysis service using FastAPI.
- * TypeScript/TSX – React frontend.
- * SQL – PostgreSQL schemas and Flyway migrations.
- * YAML – Docker Compose, application settings, and OpenAPI specifications.
- * Shell script – Docker startup and code-generation scripts.
- * JavaScript – k6 load/smoke test and frontend configuration.
- * CSS – Frontend styling with Tailwind CSS.

The main technologies are Java/Spring Boot for backend services, Python/FastAPI for scoring, and TypeScript/React for the frontend.

USER

does system have fund return

ASSISTANT

No, the system does not currently implement fund returns/refunds.

It only has a payment cancellation endpoint, but payments are immediately marked TRANSFERRED, after which cancellation is rejected. There is no refund, reversal, chargeback, or repayment-return workflow.

USER

but how do you think is it important to implements

ASSISTANT

Yes, implementing refunds is important—especially for a BNPL system.

A proper refund flow should:

- * Accept full and partial refunds.
- * Reverse or record the merchant payment.
- * Cancel unpaid installments.
- * Return money already collected from the customer.
- * Recalculate the remaining repayment balance.
- * Be idempotent to prevent duplicate refunds.
- * Store an audit trail and refund statuses.
- * Handle payment-provider failures and retries.

For this prototype, start with full refunds, then add partial refunds. This is important for demonstrating a realistic end-to-end financial lifecycle, although real production refunds require payment-provider and compliance integration.

USER

but you think will it be crucial

ASSISTANT

Yes—crucial for a production BNPL system. Without refunds, returned or cancelled purchases leave customers owing money incorrectly.

For a diploma prototype, it is not required for the basic checkout demonstration, but implementing at least full refunds would significantly improve completeness and credibility. Partial refunds can be future work.

USER

are there only netflix demo

ASSISTANT

If you mean only a frontend demo: no. The project includes working backend microservices and databases, but payments and credit data are simulated. The React website itself is mostly a static demo and isn't connected to the backend yet.

USER

can you be more detailed in project description

ASSISTANT

DETAILED PROJECT DESCRIPTION

Rocket Credit is a prototype Buy Now, Pay Later (BNPL) platform. It allows a

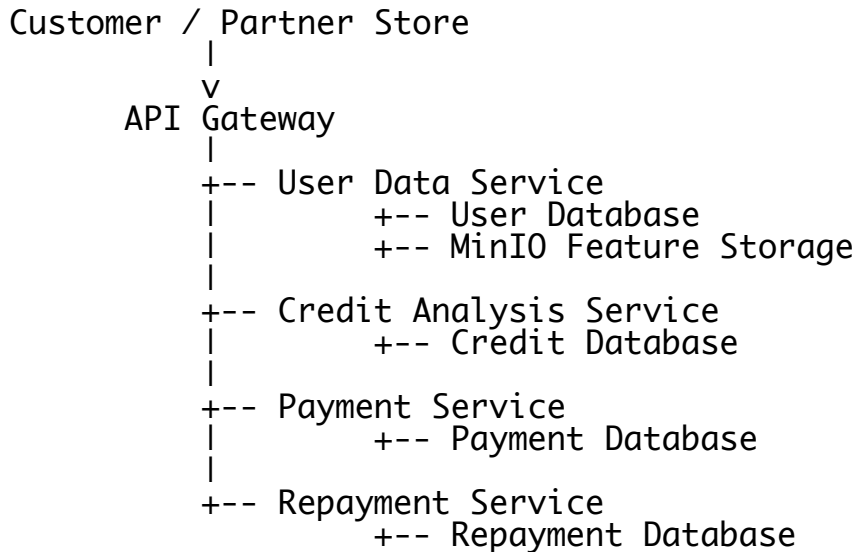
customer to purchase products from a partner store, receive an automated credit decision, pay the merchant, and repay the amount through monthly installments.

The project is designed as a diploma/research project demonstrating microservices, explainable credit scoring, financial workflows, and frontend development.

Main participants

- * Customer: requests financing and repays it in installments.
- * Partner merchant: sells products and submits checkout information.
- * Rocket Credit: evaluates the customer, pays the merchant, and collects repayments.

Overall architecture



The system uses a microservice architecture. Each major responsibility is separated into an independent application with its own database.

1. Frontend application

The frontend is built with React and TypeScript. It contains a landing page, How It Works page, business/merchant page, company information page, customer dashboard and payment calculator.

The payment calculator lets customers experiment with purchase amount, number of installments, estimated installment amount, interest amount and total repayment.

However, the frontend is currently mostly a visual demonstration. It does not call the Gateway or display real backend information. Dashboard transactions, balances and company statistics are hardcoded.

2. API Gateway

The Gateway is the main public backend entry point. The partner sends a checkout request containing partner and payment IDs, amount, currency, installment duration and buyer information.

The Gateway does not make the credit decision itself. It coordinates the other services in the correct order.

Its responsibilities include request validation, user resolution, credit-analysis requests, payment creation, repayment-plan creation, response

construction, error handling, rate limiting, idempotency, retries and circuit breaking.

Idempotency

The combination of `partnerId` and `partnerPaymentId` identifies one checkout. Redis remembers completed requests for 24 hours. If the same request is sent twice, the Gateway returns the previous result instead of creating another payment. If two identical requests arrive simultaneously, one performs the checkout while the other receives HTTP 409 Conflict.

Reliability

Calls to internal services use connection and response timeouts, automatic retry for selected User Data operations, concurrency limits, circuit breakers and downstream URL restrictions. After five relevant downstream failures, the circuit breaker temporarily stops calls to that service for 30 seconds.

3. User Data Service

The User Data Service owns customer information. It stores the internal user ID, partner-specific customer ID, name, email, KYC status, transaction history, payment-method tokens and calculated customer statistics.

When checkout begins, it finds the customer using the partner user ID and then the email address. If no customer exists, it creates one. If the customer already exists, it updates changed details.

Transaction processing

The partner can provide previous customer transactions. The service validates and stores transactions containing the date, amount and payment method. It rejects invalid entries and removes duplicates within the submitted request.

Feature calculation

The service converts customer data into credit-scoring features:

- * KYC result.
- * Orders during the last 12 months.
- * Average order value.
- * Rocket Credit on-time repayment ratio.
- * Number of late payments.
- * Number of active plans.
- * Customer tenure.
- * Estimated income.
- * Estimated credit limit.

The current calculations are simplified. The on-time ratio is hardcoded to 0.9, late payments and active plans default to zero, and income is estimated as annual transaction volume divided by 12. The credit limit is currently equal to that monthly estimate. These values demonstrate the data flow but are not suitable for real lending.

4. Claim-check component

Credit features are not sent directly through every service. User Data serializes the features into JSON and uploads them to MinIO. The Gateway passes only the returned bucket and object key to Credit Analysis, which downloads the JSON.

This is called the claim-check pattern. It helps avoid passing large or

sensitive payloads through multiple services. The shared library can calculate a SHA-256 checksum and attach correlation information.

5. Credit Analysis Service

The Credit Analysis Service is written in Python with FastAPI. It receives the customer ID, requested purchase amount and a reference to the customer's feature snapshot. It returns an approval flag, score and reason.

Eligibility checks

Before calculating the final score, it checks whether KYC passed and whether required consent exists when social analysis is enabled. Social analysis is currently disabled.

Scoring factors

```
Final Score =  
    Rocket History x 50%  
    + Partner History x 30%  
    + Purchase Amount x 20%
```

Rocket history evaluates on-time repayment, account tenure, serious late payments and active plans. Partner history evaluates order count and average order value. The amount score compares the requested amount with borrowing capacity.

```
capacity = max(creditLimit, income x 0.30)  
utilization = requestedAmount / capacity  
amountScore = 1 - utilization
```

A smaller purchase relative to capacity receives a higher score.

Decision thresholds

- * 0.60 or higher: approved.
- * 0.50-0.59: manual review.
- * Below 0.50: denied.

The request must also pass the affordability check.

Explainability

The service records the final score, decision, reason codes, individual factor scores, factor weights, thresholds used and affordability result. Example reason codes include KYC_MISSING, AFFORDABILITY_LIMIT_EXCEEDED, PARTNER_TOO_FEW_COMPLETED_ORDERS, ROCKET_RECENT_LATE_PAYMENT and ROCKET_TOO_MANY_ACTIVE_PLANS.

Dynamic policies

Weights and thresholds are stored in a versioned PostgreSQL table. Administrators can add a new policy version without changing application code. Previous decisions retain their factors, making them explainable after the policy changes.

Is it really AI?

Currently, no trained machine-learning model is used. The implementation is a deterministic, explainable scoring algorithm. Credit-bureau, social and digital footprint factors are planned or scaffolded, but they are not active. The AI-based portion is therefore primarily a future extension.

6. Payment Service

The Payment Service represents sending the approved purchase amount to the merchant. It stores payment, partner and customer IDs; amount and currency; merchant information; purchased items; provider information; payment status; failure reason; and timestamps.

Available statuses are INITIATED, TRANSFERRED, FAILED and CANCELED. The service prevents duplicates through an optional idempotency key and unique partner payment ID.

Currently, it does not connect to Stripe, a bank or another real provider. It creates a local reference and immediately changes the payment status to TRANSFERRED.

7. Repayment Service

After payment succeeds, the Repayment Service creates the customer's installment plan. It works with integer cents to ensure the installments add up exactly to the original amount.

The plan stores the plan and customer IDs, total amount, currency, installment count, provider data and plan status. Each installment stores its number, due date, amount, status, attempt count, provider charge ID, last error and last attempt time.

The first installment is scheduled one month after creation. A scheduler runs daily at 03:00 UTC, finds due installments and attempts to charge them. Failed charges return to PENDING until the retry limit is reached. After three failures the installment becomes FAILED. The current fake provider always succeeds.

8. Databases and infrastructure

The project uses separate PostgreSQL databases for users, credit analysis, payments and repayments. Database schemas are managed through Flyway for Java services and Alembic for Python.

Redis supplies rate limiting and idempotency, MinIO stores feature snapshots, and Docker Compose starts the local environment.

9. Refund support

Refunds are not implemented. A production refund workflow would accept full or partial returns, verify the original payment, return collected customer money, cancel or reduce future installments, recalculate the repayment plan, record an idempotent refund and handle provider failures.

The existing cancellation operation cannot replace refunds because payments are immediately marked TRANSFERRED, and transferred payments cannot be cancelled.

10. Planned but unimplemented features

- * Digital-footprint AI service.
- * Bank/open-banking adapter.
- * Explicit customer consent management.
- * Real payment-provider integration.
- * Refunds.
- * Notification service.
- * Kafka events.

- * Partner SDK and management portal.
- * Webhooks.
- * Customer authentication.
- * Real customer dashboard.
- * Manual credit-review interface.

Current maturity

The project is a functional backend prototype with a separate frontend demonstration. Working parts include the microservice structure, checkout orchestration, user persistence, feature generation, explainable scoring, payment records, repayment schedules, automatic fake charging, Redis idempotency and Docker deployment.

Customer financial information, scoring inputs, payment processing, installment charging, security configuration, frontend data and business statistics remain prototype-only. The next major milestone would be connecting the frontend to the Gateway, followed by a complete refund workflow and sandbox payment provider.

USER

also add languages used and for why

ASSISTANT

PROGRAMMING LANGUAGES AND WHY THEY ARE USED

Java

Used in the Gateway, User Data, Payment, Repayment and Claim Check services. Java is strongly typed and reliable for financial backend services. Spring Boot provides REST APIs, database access, validation, scheduling, security and dependency injection.

Python

Used in Credit Analysis. Python is suitable for scoring, data processing, AI and future machine-learning development. FastAPI makes it easy to expose the scoring logic as an HTTP service.

TypeScript / TSX

Used in the React frontend. TypeScript catches data-type errors while TSX combines React components with HTML-like interface markup.

SQL

Used in PostgreSQL migrations. SQL creates and updates tables, constraints, indexes, triggers and initial credit policies.

JavaScript

Used in the k6 smoke/load test. It sends simulated checkout requests and checks Gateway responses.

CSS

Used for frontend design. It defines colors, typography, buttons, forms, shadows, responsive styling and Tailwind components.

HTML

Used for the frontend entry document. It provides the browser document and root element where React is mounted.

Shell script

Used for startup and code-generation scripts. Shell scripts wait for databases, run migrations, start services and generate Java classes from OpenAPI specifications.

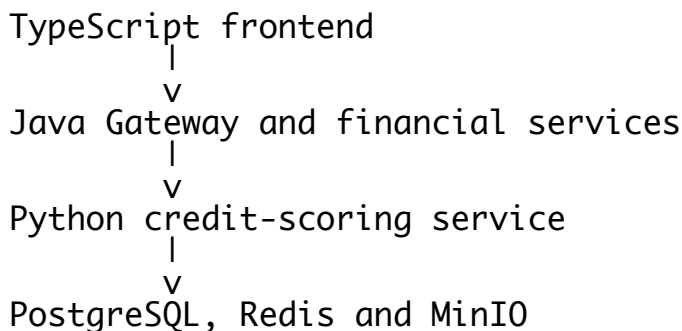
Configuration and specification formats

- * YAML: Docker Compose, Spring configuration, repayment settings and OpenAPI API definitions.
- * XML: Maven pom.xml files describing Java dependencies and builds.
- * JSON: API bodies, credit factors, reason codes and audit information.
- * Markdown: Architecture, integration, project-plan and diploma documentation.

Why the project uses multiple languages

The project follows a polyglot microservice architecture, meaning each area uses technology suited to its job:

- * Java and Spring Boot provide structured and dependable financial services.
- * Python and FastAPI make credit scoring easier to develop and extend with AI.
- * TypeScript and React provide an interactive, type-safe web interface.
- * SQL and PostgreSQL provide persistent, transactional financial records.
- * YAML and Docker connect all services into one local environment.



USER

convert this chat to pdf

END OF TRANSCRIPT